Planificador de visitas

Prueba técnica Talent Clau - Resumen técnico - 02/11/2026

Quería partir de lo que pedía el enunciado: organizar visitas para cuatro técnicos, evitar solapamientos y respetar la jornada de 08:00 a 17:00 y los 30 minutos entre obras distintas. Mi prioridad era que las reservas se guardaran correctamente y que los rechazos fueran comprensibles.

Trabajé con Codex durante el proyecto. Yo marqué los objetivos, propuse y seleccioné mejoras y decidí qué priorizar. También pedí organizar el trabajo por fases y subfases, explicar las decisiones importantes y diseñar pruebas para cada función. Codex propuso soluciones técnicas y se encargó de la programación, las pruebas automáticas y el apoyo documental. Fuimos revisando los resultados; personalmente comprobé la creación, la agenda y la reprogramación conservando las notas.

El primer tramo fue construir el núcleo. En las fases de análisis y diseño separamos los requisitos obligatorios de las mejoras opcionales y definimos los datos y las reglas. Después dividimos la preparación técnica en tres subfases: crear el proyecto, configurar las tablas y migraciones, y cargar los técnicos y obras ficticios. Utilizamos Next.js, React y TypeScript para mantener interfaz y servidor juntos, y PostgreSQL en Neon para conservar los datos.

La programación de las reservas se dividió también: primero validar los campos y la jornada; después calcular solapamientos y desplazamientos; finalmente guardar, cancelar y consultar. Separamos estas reglas de la pantalla para poder probarlas directamente y reutilizarlas. Las escrituras se protegieron con transacciones y bloqueos por técnico para evitar reservas simultáneas incompatibles. A continuación conectamos el formulario, el listado, los filtros y los mensajes. Cancelar debía liberar el horario conservando el registro. Adoptamos la hora de Madrid, sin añadir restricciones de festivos que el enunciado no pedía.

El segundo tramo fue ampliar una base que ya funcionaba. Antes repetimos las comprobaciones existentes. El nuevo plan tuvo seis fases funcionales: fichas de obra, tipos y notas; bloqueos de disponibilidad; reprogramación; búsqueda de huecos; agenda diaria; y avisos internos. El orden permitía incorporar las nuevas reglas a la búsqueda y mostrar después la planificación en la agenda.

Cada fase tenía tareas de datos, lógica, interfaz y pruebas. Por ejemplo, reprogramar exigía preparar el formulario, comprobar que la visita no hubiera cambiado desde que se abrió, validar el nuevo horario y guardar el cambio junto con su historial. Si fallaba, debía conservarse la reserva anterior y sus notas. También comprobamos que las migraciones mantuvieran las visitas existentes.

Elegí esas mejoras por su utilidad diaria: encontrar huecos, resolver ausencias, mover visitas y reunir información de cada obra. Insistí en mantener sencillo el formulario y en que las notas fueran opcionales. Los avisos quedaron dentro de la aplicación para evitar configurar un servicio de correo.

La comprobación acompañó al desarrollo. En varios bloques escribimos primero los casos de prueba y después la implementación. Al integrar cambios revisamos tanto la función nueva como las anteriores que podía afectar. Se superaron 107 pruebas unitarias y 47 de integración, además de tipos, lint y compilación. Probamos persistencia, concurrencia, conflictos y recorridos combinados por HTTP. En el navegador público se comprobó crear una visita, recargarla, abrirla y guardar una nota.

También hubo correcciones. Una validación generada por IA confundía localhost con 127.0.0.1 y rechazaba reservas legítimas. Reprodujimos el fallo, corregimos la comparación del origen y añadimos una prueba de regresión. El diario recoge esa incidencia y otras decisiones del proceso.

Para cerrar, prioricé estabilidad, documentación y publicación frente a un rediseño visual. Publicamos el código en GitHub y la demo en Vercel. Es una agenda compartida con datos ficticios y sin autenticación; los avisos requieren la aplicación abierta. Quedan la revisión completa de móvil y teclado, otros recorridos manuales y un aviso moderado en dependencias de desarrollo, documentados junto con las comprobaciones realizadas.